COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE212 - Automating Server Management	**SECTION:**	CPE31S2
DATE PERFORMED:	17/09/2026	**DATE SUBMITTED:**	19/09/2026
NAME:	Que, Josef Leroy P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 8.1 **Install, Configure, and Manage Enterprise Availability Monitor via Ansible**

1. PROCEDURE OUTPUT

1. Create a playbook that installs Nagios in both Ubuntu and CentOS. Apply the concept of creating Roles.

Environment Topology

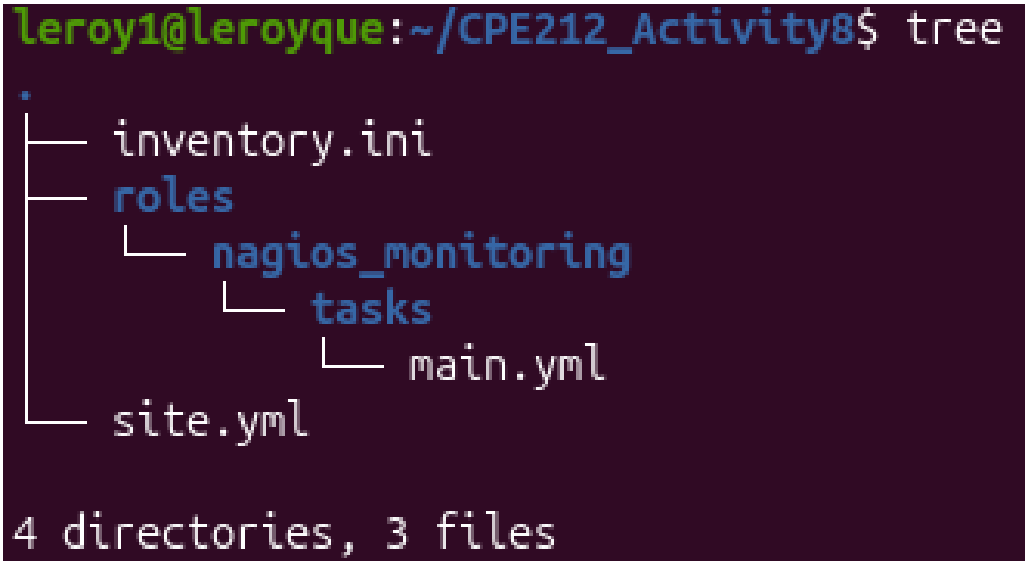
Control Node: Ubuntu Workstation

Target Node 1: Ubuntu (IP: 192.168.56.104)

Target Node 2: Ubuntu (IP: 192.168.56.106)

Target Node 3: CentOS (IP: 192.168.56.107)

Tree:



inventory.ini

```
 GNU nano 8.7.1                                                inventory.ini
[ubuntu_nodes]
ubuntu_node1 ansible_host=192.168.56.104 ansible_user=leroy1
ubuntu_node2 ansible_host=192.168.56.106 ansible_user=leroy1

[centos_nodes]
centos_node1 ansible_host=192.168.56.107 ansible_user=sysadleroy1

[all:vars]
ansible_ssh_private_key_file=~/.ssh/id_rsa
```

site.yml

```
 GNU nano 8.7.1                                                    site.yml
---
- name: Deploy Nagios Monitoring on Ubuntu and CentOS
  hosts: all
  become: true

  roles:
    - nagios_monitoring
```

2. Describe how you did step 1. (Provide screenshots and explanations in your report. Make your report detailed such that it will look like a manual.)

/roles/nagios_monitoring/tasks/main.yml

```
# STEP 1: Update Package Repository
- name: Step 1 - Update local package repository (Ubuntu)
  ansible.builtin.apt:
    update_cache: yes
  when: ansible_facts['os_family'] == 'Debian'

- name: Step 1 - Update local package repository (CentOS)
  ansible.builtin.dnf:
    update_cache: yes
  when: ansible_facts['os_family'] == 'RedHat'
```

Analysis: Uses the OS conditionals (when) to update local package caches using apt for Ubuntu and dnf for CentOS.

```yaml
# STEP 2: Install Required Dependencies
- name: Step 2 - Install build tools, web server runtime, and libraries (Ubuntu)
  ansible.builtin.apt:
    name:
      - autoconf
      - gcc
      - libc6
      - make
      - apache2
      - php
      - libapache2-mod-php
      - libgd-dev
      - extract
      - unzip
      - libssl-dev
      - openssl
      - wget
      - bc
      - gawk
      - gettext
    state: present
  when: ansible_facts['os_family'] == 'Debian'

- name: Step 2 - Enable EPEL Repository (CentOS)
  ansible.builtin.dnf:
    name: epel-release
    state: present
  when: ansible_facts['os_family'] == 'RedHat'
```

```yaml
- name: Step 2 - Install build tools, web server runtime, and libraries (CentOS)
  ansible.builtin.dnf:
    name:
      - gcc
      - glibc
      - glibc-common
      - make
      - httpd
      - httpd-tools
      - python3-passlib
      - php
      - gd
      - gd-devel
      - openssl-devel
      - wget
      - unzip
      - bc
      - gawk
      - gettext
    state: present
  when: ansible_facts['os_family'] == 'RedHat'
```

Analysis: This installs essential build tools, web runtimes, and development libraries on Ubuntu via apt and on CentOS via EPEL and dnf.

```yaml
# STEP 3: Create Service Accounts & Command Groups
- name: Step 3 - Create system group nagcmd
  ansible.builtin.group:
    name: nagcmd
    state: present

- name: Step 3 - Create system user nagios and assign to nagcmd
  ansible.builtin.user:
    name: nagios
    groups: nagcmd
    append: yes
    state: present

- name: Step 3 - Add web server user (www-data) to nagcmd group (Ubuntu)
  ansible.builtin.user:
    name: www-data
    groups: nagcmd
    append: yes
  when: ansible_facts['os_family'] == 'Debian'

- name: Step 3 - Add web server user (apache) to nagcmd group (CentOS)
  ansible.builtin.user:
    name: apache
    groups: nagcmd
    append: yes
  when: ansible_facts['os_family'] == 'RedHat'
```

Analysis: This creates the nagcmd group and nagios user, then adds the web server user (www-data on Ubuntu, apache on CentOS) to nagcmd for external command access.

```yaml
# STEP 4 & 5: Download & Extract Nagios Core
- name: Step 4 - Download Nagios Core tarball into /tmp
  ansible.builtin.get_url:
    url: "https://github.com/NagiosEnterprises/nagioscore/releases/download/nagios-4.4.14/nagios-4.4.14.tar.gz"
    dest: "/tmp/nagios-4.4.14.tar.gz"
    mode: '0644'

- name: Step 5 - Extract Nagios Core tarball
  ansible.builtin.unarchive:
    src: "/tmp/nagios-4.4.14.tar.gz"
    dest: "/tmp"
    remote_src: yes
```

Analysis: This steps downloads the Nagios Core tarball from GitHub into /tmp using get_url and extract it using unarchive.

```yaml
# STEP 6, 7 & 8: Build Nagios Core
- name: Step 6 - Run configure script with command group
  ansible.builtin.command:
    cmd: "./configure --with-command-group=nagcmd"
    chdir: "/tmp/nagios-4.4.14"
    creates: "/tmp/nagios-4.4.14/Makefile"

- name: Step 7 - Compile Nagios Core
  ansible.builtin.command:
    cmd: "make all"
    chdir: "/tmp/nagios-4.4.14"

- name: Step 8 - Install binaries, command mode, init scripts, and sample configs
  ansible.builtin.command:
    cmd: "make {{ item }}"
    chdir: "/tmp/nagios-4.4.14"
  loop:
    - install
    - install-commandmode
    - install-init
    - install-config
```

Analysis: These 3 steps run the configure script with the nagcmd group, compile Nagios Core binaries via make all, and install main binaries, command modes, init scripts, and sample configs.

```yaml
# STEP 9: Apache Web Config
- name: Step 9 - Write Apache HTTP server configuration file (Ubuntu)
  ansible.builtin.command:
    cmd: "make install-webconf"
    chdir: "/tmp/nagios-4.4.14"
    creates: "/etc/apache2/sites-available/nagios.conf"
  when: ansible_facts['os_family'] == 'Debian'

- name: Step 9 - Write Apache HTTP server configuration file (CentOS)
  ansible.builtin.command:
    cmd: "make install-webconf"
    chdir: "/tmp/nagios-4.4.14"
    creates: "/etc/httpd/conf.d/nagios.conf"
  when: ansible_facts['os_family'] == 'RedHat'
```

Analysis: This deploys the Nagios Apache web configuration using make install-webconf with OS-specific target paths and creates arguments to guarantee idempotency.

```yaml
# STEP 10: Set htpasswd Credentials
- name: Step 10 - Create HTTP basic authentication user (nagiosadmin)
  community.general.htpasswd:
    path: /usr/local/nagios/etc/htpasswd.users
    name: nagiosadmin
    password: "Password123!"
    owner: nagios
    group: nagcmd
    mode: '0640'
```

Analysis: This configures the HTTP basic authentication credentials for the nagiosadmin user, storing them in /usr/local/nagios/etc/htpasswd.users with restricted permissions.

```yaml
# STEP 11: Web Server Configuration & Start
- name: Step 11 - Enable rewrite and cgi modules (Ubuntu)
  community.general.apache2_module:
    name: "{{ item }}"
    state: present
  loop:
    - rewrite
    - cgi
  when: ansible_facts['os_family'] == 'Debian'

- name: Step 11 - Restart Web Server (Ubuntu)
  ansible.builtin.systemd:
    name: apache2
    state: restarted
    enabled: yes
  when: ansible_facts['os_family'] == 'Debian'

- name: Step 11 - Restart Web Server (CentOS)
  ansible.builtin.systemd:
    name: httpd
    state: restarted
    enabled: yes
  when: ansible_facts['os_family'] == 'RedHat'
```

Analysis: This enables the necessary Apache modules (rewrite and cgi) on Ubuntu and restarts/enables the web server service (apache2 or httpd) across both distributions.

```yaml
# STEP 12 & 13: Download & Install Nagios Plugins
- name: Step 12 - Download Nagios Plugins tarball
  ansible.builtin.get_url:
    url: "https://github.com/nagios-plugins/nagios-plugins/releases/download/release-2.4.6/nagios-plugins-2.4.6.tar.gz"
    dest: "/tmp/nagios-plugins-2.4.6.tar.gz"
    mode: '0640'
S
- name: Step 12 - Extract Nagios Plugins tarball
  ansible.builtin.unarchive:
    src: "/tmp/nagios-plugins-2.4.6.tar.gz"
    dest: "/tmp"
    remote_src: yes

- name: Step 13 - Configure, build, and install Nagios Plugins
  ansible.builtin.shell:
    cmd: "./configure --with-nagios-user=nagios --with-nagios-group=nagios && make && make install"
    chdir: "/tmp/nagios-plugins-2.4.6"
    creates: "/usr/local/nagios/libexec/check_ping"
```

Analysis: This download and extract the Nagios Plugins tarball, then configure, compile, and install the plugins into /usr/local/nagios/libexec/.

```yaml
# STEP 14: Pre-flight Check
- name: Step 14 - Execute Nagios configuration pre-flight check
  ansible.builtin.command:
    cmd: "/usr/local/nagios/bin/nagios -v /usr/local/nagios/etc/nagios.cfg"
  register: nagios_check
  failed_when: "'Total Errors:   0' not in nagios_check.stdout"
```

Analysis: This executes a pre-flight configuration check (nagios -v) to validate the Nagios setup, failing the task if any errors are detected in the output.

```yaml
# STEP 15: Enable & Start Nagios Service
- name: Step 15 - Enable and start Nagios service
  ansible.builtin.systemd:
    name: nagios
    state: restarted
    enabled: yes
```

Analysis: This uses ansible.builtin.systemd to enable and start the Nagios system service on boot.

```yaml
# STEP 16: Verify Web Interface Accessibility
- name: Step 16 - Verify Nagios Web Interface responds locally
  ansible.builtin.uri:
    url: "http://localhost/nagios/"
    user: "nagiosadmin"
    password: "Password123!"
    force_basic_auth: yes
    status_code: 200
  register: web_ui_check
```

Analysis: This uses ansible.builtin.uri to test the local web interface and verify it responds with an HTTP status code 200 using basic authentication.

3. Show an output of the installed Nagios for both Ubuntu and CentOS.

ERROR ENCOUNTER:

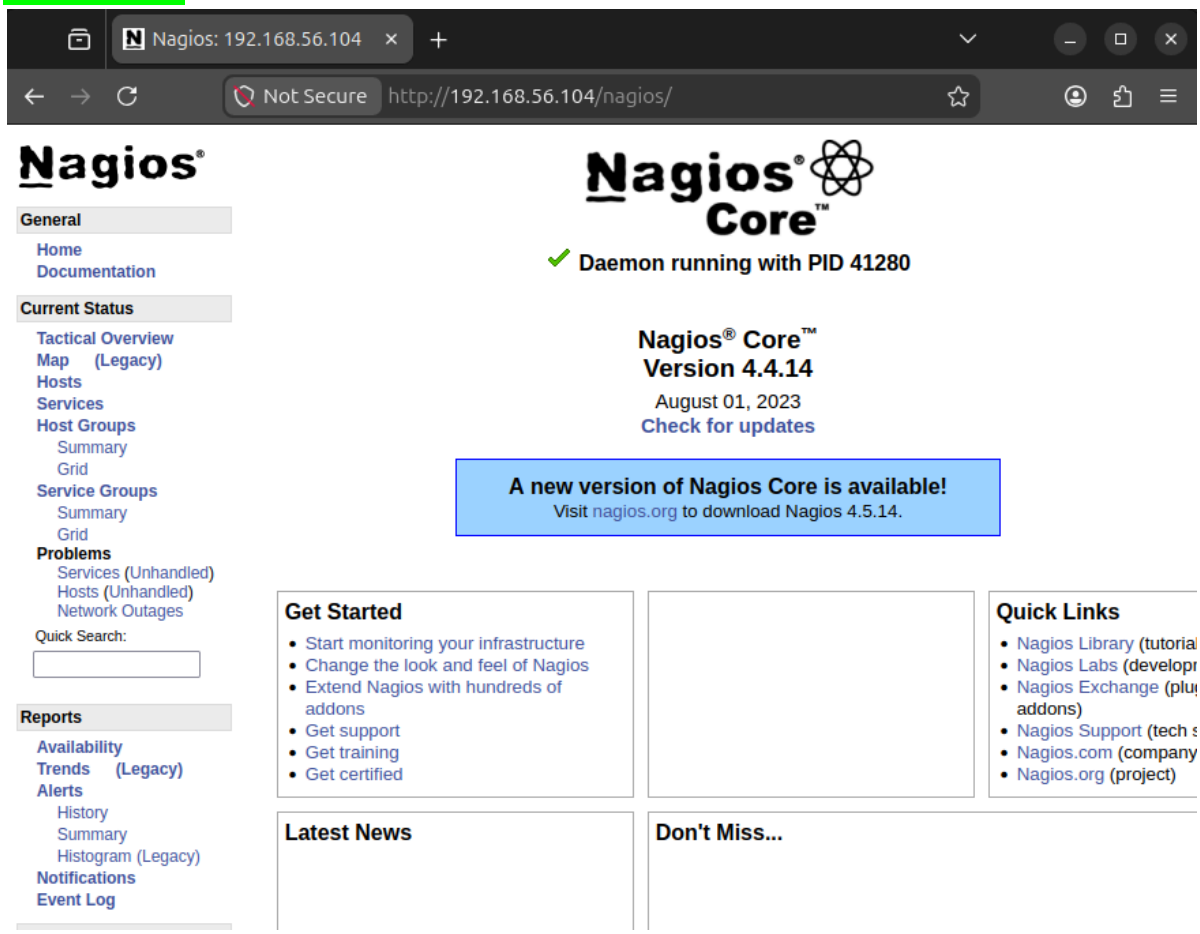
I have encountered an error during the execution of the playbook inside the class, in which is because one of the packages that is required failed to be installed on the managed node in which I have learned that the host's package repository was not updated. I then solved this by adding a task to update the package cache first and then used OS conditionals to ensure that my target packages were targeted correctly for each system. Although my intention was to show the screenshot of the error, I wasn't aware that I haven't got the chance to document it at the time, but after fixing this error, the playbook was executed smoothly afterward.

ansible-playbook -i inventory.ini site.yml –ask-become-pass

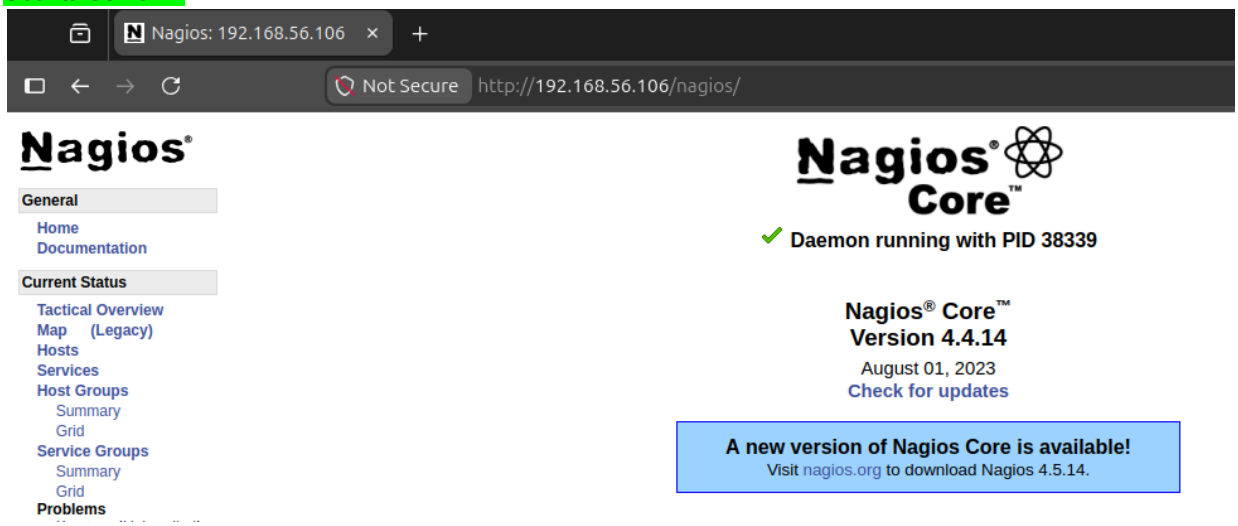
```
PLAY RECAP *********************************************************************************************************
centos_node1               : ok=21   changed=5    unreachable=0    failed=0    skipped=6    rescued=0    ignored=0
ubuntu_node1               : ok=21   changed=6    unreachable=0    failed=0    skipped=6    rescued=0    ignored=0
ubuntu_node2               : ok=21   changed=6    unreachable=0    failed=0    skipped=6    rescued=0    ignored=0
```

Analysis: The playbook execution was successful and did not produce any unnecessary errors.

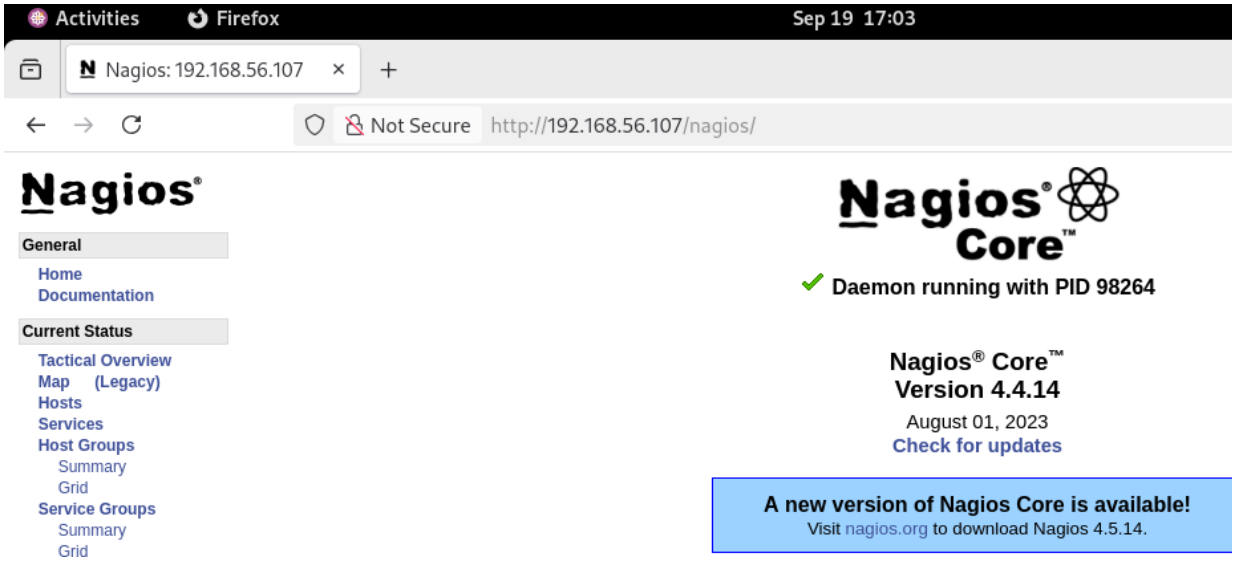
Ubuntu Server 1:



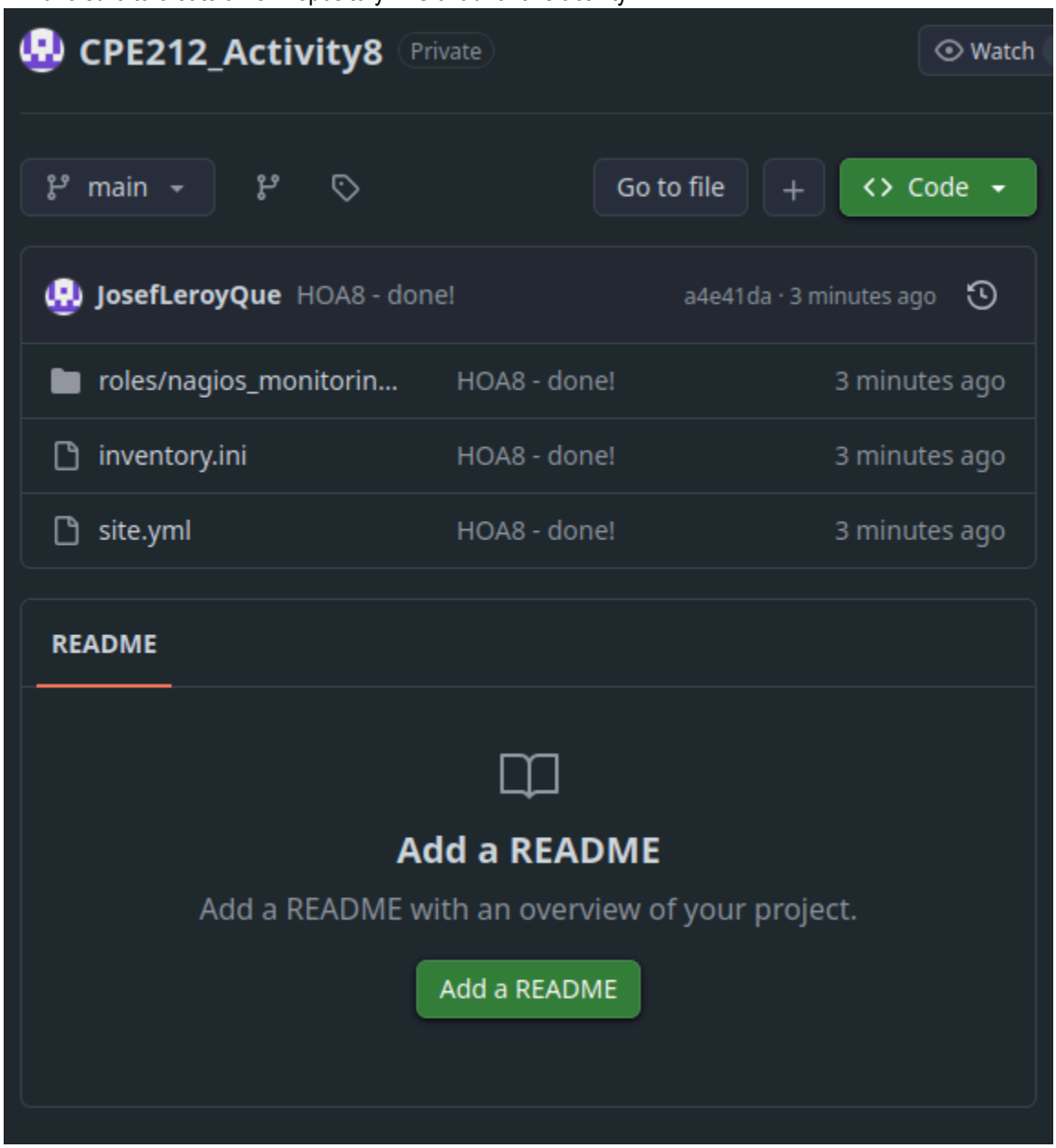
Ubuntu Server 2:



Centos:



4. Make sure to create a new repository in GitHub for this activity.



COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Answer the following:

1. What are the benefits of having an availability monitoring tool?

- With this feature, it is easier to identify when a web server goes offline and to resolve the issue without allowing users to notice it. In addition, it helps you collect data to check whether the server is working fine.

Conclusions/Learnings:

Using Ansible for configuring Nagios made me realize how simple and quick automated deployment is. I found it interesting to learn about the use of Ansible roles and operating system conditionals to install software on different types of operating systems at the same time. I was surprised with the use of commands like creates and pre-flight checks in which they help to make sure that the process is safe and no existing configurations suffer from malfunctions. The output for the configuration was verified though Nagios web dashboard that indicated that each of the three nodes were up and running.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.